

# Jev + Claude Router Playbook

Set up the two-model router that sends cheap decisions to Jev and only the hard cases to Claude.

## Why I built this

I was paying frontier prices to answer questions like "is this email an invoice or a pitch". That is not a thinking job. The answer was already one of four options before I sent the request, and I was still routing it through a full language model that writes a paragraph, charges me for every output token, and takes four seconds to do it.

Jev is a classification model from TypeSafe, launched 15 September 2026 and built by Diogo Almeida, a co-inventor of ChatGPT at OpenAI. It does not write text. You hand it a state and a set of questions you defined in advance, and it returns a typed answer plus a confidence number in 70 to 500 milliseconds. Input costs \$0.042 per million tokens and output is unmetered.

When I put Jev in front of my workflow as a router, token costs dropped 70% across 12 prompts. Nine of those 12 tasks never reached the expensive model at all. This is the exact setup, the code, the thresholds I use, and the places it broke on me first.

01

## How it actually works

1. You send Jev a state and a dictionary of questions. The state is whatever you are deciding about: a support message, a lead record, a comment, a row of JSON. The questions are typed, so the answer space is fixed before the model ever sees the input.
2. Jev answers every question in the same call, in parallel, and returns a probability distribution across your options rather than a sentence. A Choice gives you the picked option, a Score gives you a level from a rubric you wrote, and a Noul gives you a single probability between 0 and 1 for a yes/no statement.
3. Your code reads the confidence number attached to each answer and decides what happens next. High confidence runs automatically, middling confidence goes to Claude for a slower look, low confidence goes to a person. Claude only ever sees the cases that earned the cost.

02

## The one thing almost everyone gets wrong

People wire Jev in as a drop-in replacement for their language model and then wonder why the output is useless. Jev cannot generate text. There is no summary, no reply, no explanation, ever. It picks from a list you wrote. If you need words back, you still need Claude, and the whole saving comes from how rarely you call it.

### **WATCH OUT**

The second trap is ignoring the confidence field and treating every Jev answer as final. The number comes from the shape of the probability distribution, not the model's opinion of itself, so a flat spread is a real signal that the case is genuinely ambiguous. If you act on everything regardless of confidence, you will silently mis-route the hard 10% and never find out. Gate on it from day one.

---

## ROUTER.PY

```

# router.py
from typesafe_sdk import Choice, Noul, Score, TypeSafeClient

jev = TypeSafeClient() # reads TYPESAFE_API_KEY from your environment

AUTO_THRESHOLD = 0.85 # act without asking anyone
CLAUDE_THRESHOLD = 0.60 # send to Claude for a slower look

def triage(message: str) -> dict:
    """One Jev call answers all three questions in parallel."""
    r = jev.system_one(
        state=message,
        questions={
            "department": Choice(
                instructions="Which team should handle this",
                criteria={
                    "billing": "Payment, invoice or subscription issues",
                    "technical": "Bugs, integrations, anything broken",
                    "sales": "Pricing, plans or account questions",
                    "spam": "Cold pitch, phishing or automated noise",
                },
            ),
            "urgency": Score(
                instructions="How time-sensitive this message is",
                criteria=[
                    "Can wait a week",
                    "Should be handled today",
                    "Blocking money right now",
                ],
            ),
            "needs_reply": Noul(
                instructions="This message needs a written reply from a person or a
model",
            ),
        },
    )

    dept = r.answers["department"]

    if dept.confidence >= AUTO_THRESHOLD:
        return {"route": dept.choice, "handler": "auto"}
    if dept.confidence >= CLAUDE_THRESHOLD:
        return {"route": dept.choice, "handler": "claude"}
    return {"route": None, "handler": "human"}

if __name__ == "__main__":
    ticket = "Hi, I have been trying to connect my Stripe account for 3 days and the
integration keeps failing. I am losing sales. Please help ASAP."
    print(triage(ticket))

```

## RUN\_BATCH.PY

```
# run_batch.py
import csv
from concurrent.futures import ThreadPoolExecutor

from router import triage

# Jev handles roughly 10 calls a second, so run the batch in parallel.
with open("inbox.csv") as f:
    rows = list(csv.DictReader(f))

with ThreadPoolExecutor(max_workers=10) as pool:
    results = list(pool.map(lambda r: triage(r["body"]), rows))

auto    = [r for r in results if r["handler"] == "auto"]
claude  = [r for r in results if r["handler"] == "claude"]
human   = [r for r in results if r["handler"] == "human"]

print(f"{len(rows)} items")
print(f" auto-routed by Jev : {len(auto)}")
print(f" escalated to Claude: {len(claude)}")
print(f" sent to a human    : {len(human)}")
```

## ~/.ZSHRC (API KEYS)

```
# ~/.zshrc (open a new terminal after adding this)
export TYPESAFE_API_KEY="your_key_here"

# If you are going through OpenRouter instead of TypeSafe directly:
export OPENROUTER_API_KEY="your_key_here"
# model string: jev-latest (or pin a version: jev-1.13)
```

## SITES SHOWN IN THE VIDEO

- Avi Chawla's breakdown of Jev on X  
[https://x.com/ avichawla/status/2101563610644496464](https://x.com/avichawla/status/2101563610644496464)

## Set it up in Claude Code or Codex

1. Make a folder for the router and open it in Claude Code or Codex. A plain project folder is enough here, there is no skill to install. If you do want it as a reusable skill, put it in `~/ .claude/skills/jev-router/` for Claude Code, or in `.codex/skills/jev-router/` with a pointer line in your `AGENTS.md` for Codex. The folder name has to match the name: field inside `SKILL.md` or neither tool will load it.
2. Install the SDK: `pip install typesafe-sdk`. There is a JavaScript client too if you would rather run this in Node.
3. Get a key. Direct signups at `console.typesafe.ai` were paused a week after launch, so the dependable doors as of October 2026 are OpenRouter (model string `jev-latest` or `jev-1.13`) and the Vercel AI Gateway (`typesafe-ai/jev`). A new TypeSafe account comes with \$5 of credit, which is around 120 million tokens.
4. Ask Claude to add the key to your shell profile for you: tell it "add `TYPESAFE_API_KEY` to my `~/.zshrc` and tell me when to restart". Then open a brand new terminal window. The old one will not see the variable and you will spend twenty minutes debugging an auth error that is not real.
5. Paste `router.py` in and run `python router.py`. A successful first run prints a dict that looks like `{'route': 'technical', 'handler': 'auto'}` and comes back in well under a second. If it takes four seconds, you are still hitting a language model somewhere.
6. Rewrite the `criteria` dictionaries to match your own categories before you run anything real. The `criteria` text is the actual instruction to the model, so write it the way you would explain the category to a new hire, not as a one-word label.
7. Point `run_batch.py` at a CSV of 100 real items you already know the answers to. Compare Jev's routes against your own labels, then tune the two thresholds until the auto bucket is clean.

### QUICK TIPS

- Start the auto threshold high, around 0.90, and walk it down as you build trust in the categories. Moving it down is cheap, cleaning up a week of bad auto-routing is not.
- Keep the state small. The ceiling is 64,000 tokens for state and questions together, and 32,000 for any single question, so trim the email headers and signatures before you send.
- A Choice question caps at 255 options. If you are near that, your categories need collapsing anyway.

## 04

### The commands I actually run

#### FIND THE CLASSIFICATION JOBS HIDING IN MY CODE

Read every file in this project that calls an LLM. For each call, tell me whether the set of possible answers is known before the request is sent. List the ones that are, because those are classification jobs I am overpaying for. Give me the file, the line, and what the fixed answer space is.

### **TURN ONE OF THEM INTO JEV QUESTIONS**

Take the LLM call in [FILE:LINE] and rewrite it as a typesafe\_sdk system\_one call. Use Choice for the category, Score for anything graded, Noul for yes/no flags. Write the criteria as full sentences explaining each option, not single words. Keep my existing function signature so nothing downstream changes.

### **BUILD THE ESCALATION PATH TO CLAUDE**

Add confidence gating to router.py. Above [0.85] act automatically. Between [0.60] and [0.85] send the original message to Claude with the Jev answer as context and ask it to confirm or correct the route. Below [0.60] write the item to a review queue. Log which bucket each item landed in so I can count the savings.

### **CHECK MY THRESHOLDS AGAINST REAL DATA**

I have [FILE.csv] with [N] rows that I have already labelled by hand in the [COLUMN] column. Run them through router.py, compare Jev's route to my label, and show me accuracy at auto thresholds of 0.80, 0.85, 0.90 and 0.95. Tell me which threshold gives me a clean auto bucket with the fewest escalations.

### **COUNT WHAT I ACTUALLY SAVED**

Read the run log from run\_batch.py. Count how many items were handled by Jev alone and how many reached Claude. Price the Jev calls at \$0.042 per million input tokens with output free, and price what the same batch would have cost if every item had gone to [MODEL] instead. Show me the two totals and the percentage difference.

---

## When it breaks

| ERROR                                                              | FIX                                                                                                                                                                                                                              |
|--------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Auth error even though you just set the key</b>                 | You are in the terminal you had open before you edited the shell profile. Close it, open a new one, and run <code>echo \$TYPESAFE_API_KEY</code> to confirm the value is actually there.                                         |
| <b>Asking Jev to summarise or reply and getting nothing usable</b> | Jev cannot generate text at all. Any output you need in words has to come from Claude. Jev only ever picks from the options you defined.                                                                                         |
| <b>Numbers and dates coming back wrong</b>                         | Jev does not do arithmetic or counting reliably, and it cannot compare dates as ordered values. Do the maths in Python and pass the result in as part of the state.                                                              |
| <b>Request rejected for size</b>                                   | The limit is 64,000 tokens for state and questions combined, with 32,000 per individual question. Strip signatures, quoted reply chains and headers before sending.                                                              |
| <b>Choice question refused</b>                                     | A Choice caps at 255 options. Collapse your categories or split the decision into two questions.                                                                                                                                 |
| <b>Everything lands in the low-confidence bucket</b>               | Your criteria text is too thin. Each option needs a sentence explaining when it applies, not a one-word label. Rewrite the criteria and rerun the same batch.                                                                    |
| <b>Costs did not drop much</b>                                     | Check how many items are escalating. If most of the batch is reaching Claude anyway, the categories overlap and Jev cannot separate them, so the answer space needs rethinking rather than the threshold.                        |
| <b>Cannot create a TypeSafe account</b>                            | Direct signups were paused shortly after the September 2026 launch. Go through OpenRouter with <code>jev-latest</code> , or the Vercel AI Gateway with <code>typesafe-ai/jev</code> , neither of which needs a TypeSafe account. |

## Ready to go deeper?

This playbook is the short version. Inside the hub you get the full build walkthroughs, the prompt library, and the systems I use to ship this in a weekend.

**Join the Build →**

<https://hub.digicuratoragency.com/join>